

Emotion Learning in Large Language Models

Implementation and Specification Audit

Extraction6 v6.2 — documented and corrected implementation — technical architecture, compliance analysis, and implementation report

Amirali Famili • Student ID: 720060845/1

Executive Summary

The supplied Extraction6 implementation is best understood as the experimental infrastructure and representation-extraction layer of the final-year project, rather than the complete probing experiment. It already implements a strong set of reproducibility, model-loading, memory, persistence, and runtime-observability mechanisms. In particular, the code freezes experiment configuration, pins model revisions, validates checkpoint loading, separates core from optional loading discrepancies, records runtime telemetry, and persists pooled hidden states in memory-mapped arrays. These features substantially strengthen the reliability of the eventual probing study.

The implementation does not yet satisfy the full project specification by itself. The specification explicitly requires lightweight classifier probes trained on the layer-wise representations, evaluation with metrics such as accuracy and F1-score, confusion matrices, interpretation of which layers encode emotion, ablation studies, and preservation of both token-level and overall representations. None of the probe-training or evaluation stages are present in this module. The extractor also does not currently persist labels and train/validation/test splits alongside the representation arrays, creating a downstream provenance risk. Therefore, the correct claim is: Extraction6 successfully implements the representation-acquisition foundation required for the project, but the final research pipeline remains incomplete until the downstream probing and analysis stages are implemented.

1. Source and scope of this audit

This audit uses the complete Extraction6 v6 source supplied in the conversation and the full textual project specification reproduced in the same request. The conversation file index did not expose a separate PDF attachment for direct page-image inspection, so the analysis can verify the specification text itself but cannot independently verify PDF-only layout details, figures, signatures, or visual formatting. The technical requirements quoted and assessed here are therefore those present in the supplied specification text.

2. Project objective expressed as an executable pipeline

The specification asks whether a pretrained language model—primarily Qwen—contains emotion-related information in its internal representations and, critically, at which layers that information becomes linearly or non-linearly recoverable. The complete research pipeline therefore has two distinct halves: representation acquisition and probing/analysis.

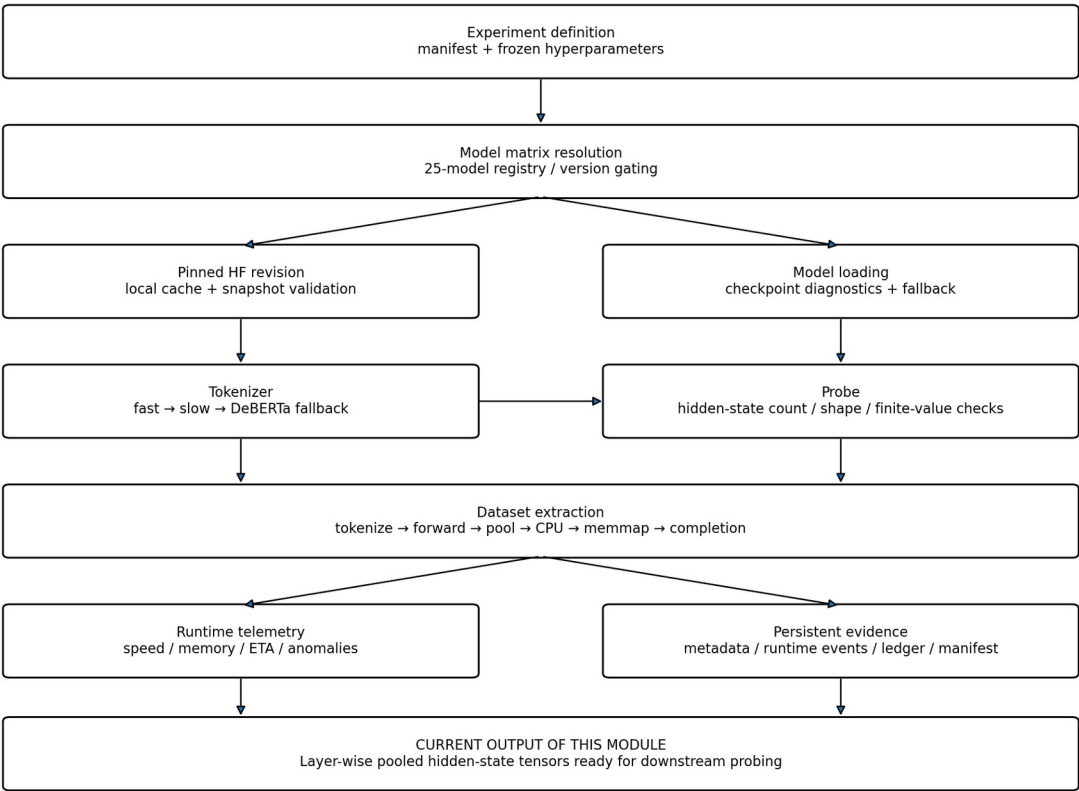


Figure 1. Actual Extraction6 execution pipeline and the boundary between the current extraction module and the downstream probing stage.

3. Specification compliance verdict

Specification item	Status	Assessment
Primary Qwen target	Yes, with extension	Qwen2 and Qwen3 families are in the registry; the matrix is broader than the specification.
GoEmotions + ISEAR	Yes	The extractor accepts both datasets and preserves dataset-level provenance, but labels are not exported by this module.
Same pretrained tokenizer	Yes	The tokenizer is loaded from the exact model snapshot and

		is used for extraction.
Layer-wise hidden states	Yes	All returned hidden states are extracted and pooled; N+1 states are expected for N transformer layers.
Overall representation	Yes	Mean/first-token/last-token pooled representations are written to memmap.
Token-level representations	No	Only pooled tensors are persisted. Full token-level storage is not implemented.
Classifier probe	No	No linear or feed-forward probe training is present.
Layer-wise evaluation	Not yet	The extraction output is suitable for it, but evaluation code is absent.
Accuracy/F1/confusion matrices	No	No probe metrics or confusion matrices are produced here.
Ablation studies	No	No keyword/emotion removal pipeline is implemented.
Alternative models	Extension	The 25-model matrix is a useful comparative extension, but not itself part of the core Qwen objective.
Reproducibility	Strong foundation	Pinned revisions, frozen hyperparameters, provenance metadata and persistent ledgers are implemented.
Runtime forensic observability	Strong	Speed, memory, batch timing and anomaly evidence are persistently logged.

Overall judgement: the module points strongly and correctly toward the specified research objective, but it should be presented in the dissertation as the extraction/infrastructure stage, not as the completed probing experiment.

4. What the code actually does

4.1 Experiment identity and immutability

HyperParameters is the frozen experiment contract. The configuration contains pooling strategy, maximum sequence length, storage dtype, CPU/accelerator dtype policy, per-model frozen batch sizes, thread settings, SDPA policy, flushing policy, download retry policy and environment-fingerprint enforcement. Its `config_hash` provides an immutable identifier for the exact experiment configuration.

When `run_experiments` or `run_model_matrix` finds an existing `run_manifest.json`, that manifest becomes authoritative. This is an important scientific safeguard: a later notebook call cannot silently change pooling, sequence length or batch size and then mix the resulting representations with an earlier experiment.

4.2 Model matrix

The ModelSpec registry contains 25 models across six groups: classical encoders, early decoders, tiny modern models, Qwen scaling models, independent small models and stretch models. The project specification originally proposes Qwen as the principal subject and BERT as a possible comparison; the current implementation broadens this into a controlled comparative matrix.

4.3 Hugging Face snapshot preparation

Each model is associated with a pinned Hugging Face commit SHA. Repository contents are inspected through `list_repo_files`, the download pattern selects PyTorch-compatible checkpoint files, `snapshot_download` retrieves them into the external SSD cache, and `_validate_snapshot` verifies the presence of `config.json` and usable checkpoint material.

4.4 Model loading and forensic validation

The loader requests `output_loading_info` from Transformers so missing, unexpected and mismatched keys are not inferred only from warnings. The loading report is divided into core discrepancies and non-core discrepancies. Optional poolers containing meta tensors can be disabled because this project requires hidden-state representations, not pooled classification heads. A small probe is then executed to verify hidden-state count, tensor rank, sequence dimension, hidden size and finite numerical values before a dataset is allowed to run.

4.5 Dataset representation extraction

Each dataset is resolved to a text column, converted to strings, fingerprinted, and associated with an output directory. The representation array is a NumPy memory-mapped array with shape approximately (number of samples, number of hidden states, hidden size). For example, BERT with 12 transformer layers yields 13 stored states per sample. Each batch is tokenized, transferred to the execution device, forwarded through the frozen model, pooled at every returned hidden state, converted to CPU float32, written to the memmap and accompanied by a completion flag.

4.6 Runtime telemetry

RuntimeReporter divides observability into layers. INFO exposes progress, throughput, ETA, last-batch timing, memory and rolling performance. VERBOSE exposes forensic diagnostics. DEBUG exposes individual batch events. The logger additionally persists runtime events and snapshots, so the research record survives even when a notebook cell or terminal view is lost.

5. Detailed code reference

The following sections document every constant, class and function in the supplied module. The purpose is to make the codebase readable as a research instrument rather than merely as executable software.

5.1 Global constants and variables

Variable	Type	Meaning
EXTERNAL_MOUNT	Path	Root mounted external SSD (/Volumes/Amirali). All persistent experiment data is anchored here.
EXTERNAL_ROOT	Path	External hidden-state project root.
HF_ROOT	Path	Hugging Face cache root on the external SSD.
HF_HUB_CACHE	Path	Hugging Face model snapshot cache.
HF_XET_CACHE	Path	Xet cache used by Hub transfers when applicable.
HF_ASSETS_CACHE	Path	HF auxiliary assets cache.
EXPERIMENTS_ROOT	Path	Root directory containing experiment manifests and ledgers.
HYPERPARAMETER_CONFIG_PATH	Path	Global default hyperparameter JSON file.
MODEL_REVISION_MANIFEST	Path	Persistent map from model id to pinned commit SHA.
MEASUREMENT_LEDGER	Path	Append-only JSONL ledger of completed/failing measurement records.
DEFAULT_POOLING / DEFAULT_MAX_LENGTH	str/int	Default representation policy: masked mean pooling and 512-token maximum sequence length.
DEFAULT_STORAGE_DTYPE	str	Storage precision for the memmapped output arrays.
DEFAULT_FLUSH_EVERY_BATCHES / DEFAULT_FLUSH_SECONDS	int/float	Durability cadence for memmaps and runtime state.
MEMORY_RESERVE_FRACTION / MIN_MEMORY_RESERVE_GB / MAX_MEMORY_RESERVE_GB	numeric	Parameters for reporting an adaptive safety reserve.
MAX_MODEL_MEMORY_FRACTION	float	Preflight threshold applied against total RAM; intentionally conservative but not an activation-memory model.
MODEL_LOAD_SAFETY_FACTOR	float	Multiplier applied to estimated loaded weights to approximate model-loading peak.

RUNTIME_REPORT_INTERVAL_*	numeric	Controls how often speed telemetry is emitted based on time or completed samples.
DOWNLOAD_*	numeric	Retry, concurrency and HTTP timeout policy for Hub downloads.
USE_LOW_CPU_MEM_USAGE / USE_SDPA / STRICT_META_VALIDATION	bool	Primary loader policies.
TEXT_COLUMN_CANDIDATES / TEXT_COLUMN_EXCLUDE	tuple/set	Heuristic text-column resolver configuration.
COMMON_MODEL_FILES / UNNECESSARY_FRAMEWORK_PATTERNS	list	Hugging Face file selection policy.
NON_CORE_MODEL_PARAMETER_PREFIXES	tuple	Prefixes considered non-core for discrepancy classification.

5.2 ModelSpec and registry

ModelSpec is a frozen dataclass describing one experiment model. name is the Hugging Face repository id; family is the human-readable family label; generation identifies the model generation; parameter_billions is a registry estimate; architecture distinguishes encoder/decoder; training_status records the intended interpretation of the checkpoint; group controls matrix organisation; batch_hint supplies the frozen starting batch; pooling/max_length are model-specific defaults; min_transformers gates known incompatible model families; trust_remote_code controls Hub code loading; gated marks access-controlled models; role identifies primary versus auxiliary use.

MODEL_REGISTRY is the 25-model scientific matrix. MODEL_BY_NAME provides constant-time lookup. GROUP_ORDER fixes the display and execution order. get_model_specs optionally filters the matrix by group. get_model_spec returns a custom fallback ModelSpec for non-registered ids, although the fallback currently uses a synthetic 1B parameter estimate and should therefore be treated as a diagnostic convenience rather than a reliable memory estimate.

5.3 System and utility functions

Function	Purpose
_separator	Returns a visual console separator of configurable width.
_header	Prints a boxed section header.
_print_verbose / _print_info / _print_critical	Level gates for the three user-visible logging layers.
_verbosity_name	Reports the highest active visibility level.
_format_duration	Converts seconds into human-readable seconds/minutes/hours.
_truncate_traceback	Caps forensic traceback length to keep logs bounded.

save_json	Performs atomic JSON replacement using a temporary file.
append_jsonl	Appends one JSON record to a persistent event/measurement ledger.
stable_hash	Produces a deterministic SHA-256-derived short identifier from JSON-serialisable content.
_bytes_to_gb	Converts bytes to GiB using 1024^3 .
_sysctl_int	Reads an integer from macOS sysctl.
get_memory_info	Returns total, available and used RAM, using psutil where available.
get_external_storage_usage	Reads disk capacity and free space for the external mount.
get_cpu_info	Captures platform, CPU count, Python and PyTorch threading settings.
get_process_rss_gb	Reads current process resident memory.
adaptive_memory_reserve_gb	Computes a conservative adaptive RAM reserve for diagnostics.
system_fingerprint	Builds a reproducibility fingerprint from platform/runtime characteristics.
get_best_device	Selects CUDA, then MPS, then CPU.
model_device	Reads the device of the first model parameter without moving the model.
synchronize_device	Synchronises CUDA/MPS before timing boundaries.
cleanup	Runs garbage collection and device cache cleanup.
dtype_from_name	Maps a textual dtype name to a PyTorch dtype.
print_system_report	Prints the initial system/storage diagnostic panel.
configure_cpu_threads / configure_cpu_interop_threads	Applies requested PyTorch CPU threading settings.

5.3 Runtime diagnostics

Function	Purpose
runtime_diagnostic_snapshot	Builds a full-point-in-time diagnostic record combining stage, provenance, versions, memory, storage and CPU state.
write_runtime_diagnostic	Persists a diagnostic snapshot using save_json.

5.3 Experiment configuration

Function	Purpose
default_hyperparameters	Constructs the default frozen configuration from the registry batch hints.
ensure_hyperparameter_file	Creates the default JSON configuration if none exists.

load_hyperparameters	Loads and validates the configuration file.
experiment_root	Creates and returns the experiment directory.
load_existing_experiment_parameters	Loads the authoritative run manifest configuration on resume.
resolve_experiment_parameters	Either creates a new experiment configuration or returns the existing frozen configuration while recording requested overrides.
write_experiment_manifest	Creates or validates the experiment manifest and refuses matrix/hash/environment drift.

5.3 Hugging Face/cache

Function	Purpose
verify_external_drive	Fails early if the external storage path is unavailable.
configure_external_storage	Creates cache directories and sets HF environment variables.
verify_huggingface_storage	Confirms required cache directories exist.
load_revision_manifest / save_revision_manifest	Reads/writes the model-to-commit-SHA map.
get_or_pin_model_revision	Obtains a commit SHA once and reuses it thereafter.
_repository_files	Lists repository files for a specific revision.
_download_patterns	Chooses PyTorch checkpoint file patterns and excludes foreign frameworks.
_is_transient_download_error	Classifies retryable Hub/network failures.
_snapshot_download_compat	Provides a version-compatible wrapper around snapshot_download.
prepare_model	Orchestrates revision pinning, file discovery, download retries, snapshot validation and timing.

5.3 Model architecture/preflight

Function	Purpose
checkpoint_inventory	Enumerates safetensors, .bin and checkpoint index files with sizes.
model_weight_bytes	Estimates active checkpoint size, preferring safetensors over stale .bin files.
_validate_snapshot	Checks the local snapshot is within the external root and contains usable checkpoint material.
get_model_num_layers	Extracts layer count from common configuration field names.
get_model_hidden_size	Extracts hidden dimensionality from common configuration field names.
checkpoint_dtype_from_config	Maps configuration dtype metadata to a torch dtype.

estimate_loaded_weight_gb	Estimates loaded parameter memory after dtype conversion.
preflight_model	Builds architecture/checkpoint/memory diagnostics and decides whether the model crosses the configured coarse memory threshold.

5.3 Model loading/probing

Function	Purpose
has_meta_tensors	Enumerates parameters/buffers still on the meta device.
_accelerate_available	Checks whether accelerate is installed for low-memory loading.
get_forward_input_keys	Inspects model.forward so only accepted tokenizer fields are passed.
prepare_model_inputs	Moves accepted input tensors to the target device.
_normalise_key_list	Normalises loading-info lists to string lists.
classify_model_loading_issues	Separates core versus non-core checkpoint discrepancies and meta tensors.
classify_loading_info	Compatibility alias for the same classifier.
disable_optional_meta_modules	Disables non-essential poolers when they contain meta tensors.
_load_model_from_snapshot	Calls AutoModel.from_pretrained with loading diagnostics and compatibility fallback.
run_model_probe	Executes a minimal real forward pass and validates hidden-state geometry and finite values.
load_model	Runs primary loading, probe validation, optional fallback, timing and diagnostic aggregation.
load_model_config	Loads AutoConfig from the local snapshot.
load_tokenizer	Loads fast tokenizer, then slow tokenizer, then an explicit DeBERTa SentencePiece fallback.

5.3 Dataset/storage

Function	Purpose
_column_diagnostic	Inspects sample values in a candidate dataset column.
detect_text_column	Selects a text column from known names or a conservative string-content heuristic.
dataset_texts	Converts the selected dataset column into a list of strings.
dataset_fingerprint	Uses the dataset-native fingerprint when available, otherwise a derived head/tail hash.

dataset_signature	Builds stronger provenance metadata including native fingerprint, text column, sample count and sample hashes.
build_dataset_storage_paths	Creates the data/metadata directory tree and standard output filenames.
build_model_directory / build_dataset_directory	Build deterministic model/dataset output locations.
flush_array	Flushes a NumPy memmap to disk when applicable.
_metadata_matches_completion_signature	Checks whether an existing completed dataset belongs to the same experiment/provenance contract.
dataset_output_is_complete	Requires states, completion map, all completion flags and matching metadata before declaring completion.
all_model_datasets_complete	Checks every requested dataset before loading a model.

5.3 Extraction

Function	Purpose
pool_hidden_states	Converts all token-level hidden states in a batch into layer-wise pooled representations.
is_oom_error	Recognises common memory-exhaustion exceptions without changing the configured batch.
_find_first_incomplete	Locates the first unfinished sample in a completion map.
extract_dataset	Core representation-extraction function: provenance, memmap allocation, resume, tokenisation, forward pass, layer-wise pooling, validation, durable writes, runtime telemetry and final metadata.

5.3 Execution/orchestration

Function	Purpose
write_model_metadata	Persists model-level provenance and loading diagnostics.
check_transformers_version	Gates models against registry-declared minimum Transformers versions.
_prepare_environment	Selects device, applies thread settings and captures the environment fingerprint.
_run_resolved_experiment	Executes the already-resolved experiment across models and datasets; handles failures and cleanup.
run_experiments	Public API for running an explicit model list

	after resolving configuration exactly once.
run_model_matrix	Public API for executing the registry-driven model matrix.
summarize_measurement	Flattens the principal performance fields of a measurement record for later analysis.
record_hyperparameter_measurement	Adds hyperparameter/measurement pairs to the append-only experiment ledger.

5.4 Corrected interpretation of matrix entries 10-12

The three entries that previously caused disproportionate confusion are now treated explicitly. SmolLM2-360M is a 32-layer Llama-architecture model with hidden size 960; the current registry uses a frozen batch hint of 8 for an 8-GiB CPU extraction because retaining all 33 hidden states at 512 tokens creates substantial activation memory. The official model card describes SmolLM2-360M as the 360M member of the SmolLM2 family. (Hugging Face model card and configuration for SmolLM2-360M, checked 23 August 2026).

Gemma 3 270M is structurally different from the earlier decoder-only entries: the current configuration exposes 18 hidden layers and a 640-dimensional hidden state. The model is gated in the registry and is version-gated so an older Transformers installation is skipped rather than loaded optimistically. Current Hugging Face configuration examples expose the Gemma 3 text configuration with hidden size 640 and 18 layers. (Hugging Face Gemma 3 configuration sources, checked 23 August 2026).

Qwen2-0.5B is a 24-layer Qwen2 decoder with hidden size 896. The official configuration records bfloat16 weights and 24 hidden layers; when this project deliberately converts to CPU float32 for deterministic extraction, the parameter footprint approximately doubles relative to the bfloat16 checkpoint, before activation memory is considered. The frozen batch hint is therefore reduced to 8 in the corrected matrix. (Hugging Face Qwen2-0.5B model card and configuration, checked 23 August 2026).

Entry	Corrected matrix interpretation	Reason
10 — SmolLM2-360M	360M-class; 32 layers; hidden size 960; frozen batch 8	All-layer hidden-state retention creates much more activation memory than parameter count alone suggests.
11 — Gemma 3 270M	268M-class; 18 layers; hidden size 640; frozen batch 16; version-gated	Newer Gemma 3 architecture requires an adequately recent Transformers implementation.
12 — Qwen2-0.5B	0.5B-class; 24 layers; hidden size 896; frozen batch 8	CPU float32 loading plus all-layer activations makes the former batch-32 setting unsafe on an 8-GiB machine.

6. Runtime data model and persistent outputs

Artifact	Purpose	Research significance
run_manifest.json	Frozen experiment configuration and matrix identity.	Prevents silent configuration drift during long-running experiments.
hyperparameters.json	Readable copy of the frozen parameter set.	Human-auditable experiment contract.
environment.json	Runtime/platform/device information.	Records the computational environment.
model_metadata.json	Model revision, preflight, load/probe diagnostics.	Connects representations to an exact model checkpoint.
hidden_states.npy	Memory-mapped pooled representations.	Primary data product consumed by probing.
completed.npy	Boolean completion map per sample.	Supports restart/resume without discarding completed work.
extraction.json	Final dataset performance/provenance record.	Links data, code settings and timing.
runtime_state.json	Latest runtime or final measurement state.	Operational record for crashes and performance anomalies.
runtime_events.jsonl	Detailed append-only runtime events.	Forensic timeline for individual batches/errors.
measurement_ledger.jsonl	Cross-experiment measurement ledger.	Supports later performance/hyperparameter analysis.

7. Exact relationship to classifier probing

The output representation can be conceptualised as $X[\text{layer}, \text{sample}, \text{feature}]$. A linear probe for a categorical emotion dataset would learn W_{layer} and b_{layer} such that $\text{softmax}(X_{\text{layer}} W_{\text{layer}} + b_{\text{layer}})$ predicts the emotion labels. Repeating this independently for every layer gives a layer-by-layer recoverability curve. The extractor is therefore the part of the pipeline that creates X ; the probe and evaluation stages must create the curves and evidence needed to answer the research question.

Representation matrix for one dataset: $X \in \mathbb{R}^{(N \times L \times H)}$
where N =samples, L =hidden states, H =hidden size.

For each layer l : $\text{probe}_l(X[:, l, :]) \rightarrow \text{emotion label}$
Then compare accuracy, macro-F1, per-class F1, confusion matrices and confidence across l .

8. Important specification gaps

Gap	Priority	Required action
-----	----------	-----------------

No probe classifier is implemented.	Critical	Add a separate probing module that consumes the memmaps and trains a linear classifier first, followed by an explicitly controlled non-linear probe only if justified.
No label/split artifact is emitted.	Critical	Persist sample index, original label(s), dataset split and a stable row-order identifier next to every extracted dataset.
No token-level representations are persisted.	High	The specification requests both token-level and overall representations. Full token storage is prohibitively large for all models; use a documented subset/stratified token-level store or a configurable token-level mode.
No accuracy/F1/confusion matrix analysis exists.	Critical	Implement layer-wise train/validation/test evaluation, macro-F1, weighted-F1, per-class metrics and confusion matrices.
No ablation study exists.	High	Implement controlled lexical masking/removal and/or label-focused ablations, with exactly matched evaluation protocols.
No split/reproducibility lock for probing exists.	High	Freeze train/validation/test indices and random seeds in an experiment manifest.
Mean/first/last pooling policy is available but last/first token pooling is padding-sensitive.	High	Select the first/last valid token using attention-mask positions, not absolute positions 0 or sum(mask)-1.
Preflight ignores batch activation and output_hidden_states memory.	High	Estimate activation cost; this is especially important because the extractor requests every layer's token-level hidden state before pooling.
Cached models still depend on Hub file discovery before download validation.	Medium	Prefer a validated local pinned snapshot before attempting network discovery on resume.
Runtime anomaly counting can re-count the same last batch.	Medium	Track the batch identifier already analysed by the anomaly detector.

Final runtime state is overwritten by extraction metadata.	Medium	Keep runtime_state.json as a runtime-state schema and store final extraction metadata separately.
Current use_half_precision flag has no effect on CPU inference.	Medium	Make this explicit in the report and diagnostics; optionally define a CPU policy such as “FP32 forced for CPU”.

9. Memory and performance implications

A major design fact is that `output_hidden_states=True` retains every layer’s sequence representation for the batch. This means the apparent model parameter size is not the only memory driver. The batch activation itself can be substantial. For illustration, a BERT-style configuration with batch 64, sequence length 512, 13 hidden states and hidden size 768 requires approximately 1.22 GiB just for float32 hidden-state tensors before inputs, model parameters, temporary attention buffers, tokenizer objects and Python overhead. A Qwen2-0.5B-like configuration with batch 32, sequence length 512, 25 states and hidden size 896 is approximately 1.37 GiB for those hidden states alone. This explains why a small model can still be slow or memory-heavy when all layers are retained.

- On CPU, the forward pass is the dominant cost in the existing telemetry; tokenizer, pooling and memmap writes are typically much smaller except during anomalous batches.
- The frozen batch policy is scientifically useful because it prevents runtime adaptation from changing the conditions under which models are compared.
- For maximum speed, the next optimisation should reduce unnecessary full-sequence hidden-state retention while preserving the exact per-layer pooled representation needed by the probes.

10. Novel engineering techniques and contributions

These features should be described as engineering contributions unless they are experimentally validated as a novel research method. The code itself does not yet establish scientific novelty in emotion probing.

Technique	Why it is useful
Frozen experiment manifest with hash	Turns a long-running extraction into an immutable experimental contract.
Pinned model revision matrix	Prevents silent model drift between runs and makes cross-model comparisons reproducible.
Forensic loading classification	Separates harmless task-head/pooler discrepancies from missing core representation weights.
Probe-before-extraction validation	Prevents wasting hours extracting a dataset

	with a model that cannot produce valid hidden states.
Completion map + provenance signature	Allows restart/resume while protecting against accidental mixing of incompatible datasets/configurations.
Layered observability	Separates scientific speed/state information from verbose forensic information and per-batch DEBUG output.
Persistent measurement ledger	Creates a machine-readable history of runs for later performance and hyperparameter analysis.
External-cache architecture	Separates model/hidden-state storage from the primary system volume, improving capacity management.
Explicit no-runtime-batch-change rule	Protects the comparability of probing experiments at the cost of adaptive performance.

11. Recommended final architecture to satisfy the specification

- Extraction stage: retain the current module as the controlled representation-acquisition engine.
- Label/split stage: write aligned labels, row IDs and train/validation/test indices to a frozen dataset manifest.
- Probe stage: first train independent linear probes on every layer using the pooled representations.
- Validation stage: evaluate on held-out data and record accuracy, macro-F1, weighted-F1, per-class F1 and confusion matrices.
- Non-linear extension: only introduce a small MLP when the linear probe fails in a way that justifies the additional capacity; record probe architecture and random seed as part of the experiment hash.
- Ablation stage: remove or mask controlled lexical cues and rerun the same frozen probe protocol.
- Cross-model extension: use BERT/Qwen2/etc. as comparative experiments after the principal Qwen result is complete.
- Interpretation stage: identify the first layer at which emotion information becomes recoverable, the layer of peak recoverability, and whether late layers preserve, compress or transform that signal.

12. Proposed downstream probing outputs

File	Content
probe_manifest.json	Frozen probe architecture, seed, split IDs, input representation, layer list.
layer_metrics.csv	One row per model × dataset × layer with

	accuracy/F1/other metrics.
confusion_matrix_layer_XX.png	Visual confusion matrix for each selected layer.
layer_curve.png	Accuracy and macro-F1 versus layer depth.
ablation_results.csv	Controlled ablation comparison.
probe_summary.json	Peak layer, early-emergence layer, uncertainty/variance and protocol summary.

13. Suggested report structure aligned with the specification

- 1 Introduction — motivation, black-box framing, emotion representation question.
- 2 Emotion Models — categorical versus dimensional emotion representations.
- 3 LLMs — tokenisation, transformers and Qwen architecture.
- 4 Interpretability and Explainability — classifier probing, linear/non-linear probes and limitations.
- 5 Project Specification — research question, datasets, controlled pipeline and hypotheses.
- 6 Implementation — Extraction6 architecture, reproducibility controls, hidden-state storage and runtime instrumentation.
- 7 Experiments — Qwen primary results, layer-by-layer probes, controls, split methodology and ablations.
- 8 Comparative Study — Qwen2/BERT/other registered models if retained as an extension.
- 9 Discussion — where emotion information becomes recoverable, limitations of probe-based conclusions, dataset effects and alternative explanations.
- 10 Conclusion — direct answer to whether and where emotional information is encoded/recoverable.

14. Claims the current code can and cannot support

Claim	Can the current module support it?
The model checkpoint is pinned and reproducible.	Yes, subject to the environment and external cache being preserved.
Representations can be extracted layer-wise.	Yes, for the model families that successfully load and pass the probe.
The model has learned emotion.	No. Extraction alone does not establish this.
Emotion appears at layer N.	No. Requires probe evaluation across layers.
Qwen is better than BERT for emotion representation.	No. Requires identical probing protocols and statistical comparison.
A representation contains task information.	Only as a hypothesis until a properly controlled probe demonstrates recoverability.

14.1 External verification notes

Model-entry corrections in this report were cross-checked against the current Hugging Face model cards/configuration pages for SmolLM2-360M, Gemma 3 270M and Qwen2-0.5B on 23 August 2026.

The report uses these sources only to verify model architecture/size metadata; they do not alter the experimental protocol or constitute experimental results.

15. Final assessment

The current code is a strong foundation for the project because it turns model probing from an ad-hoc notebook procedure into a controlled, resumable and heavily instrumented data-production pipeline. Its strongest contribution is experimental discipline: exact model revisions, frozen hyperparameters, explicit device and batch policy, provenance-aware resumption, checkpoint diagnostics, probe-before-extraction validation, and persistent runtime evidence. These characteristics are unusually valuable for a final-year project where thousands of batches and many model variants can otherwise make the final dataset difficult to audit.

The key distinction for the dissertation is between representation availability and scientific evidence. Extraction6 creates the representations on which the scientific test will operate; it does not itself test the emotion-learning hypothesis. The next implementation milestone should therefore focus less on adding additional models and more on building the layer-wise linear-probing experiment, frozen dataset splits and label provenance. Once those are in place, the existing runtime and provenance infrastructure will make the resulting conclusions substantially easier to defend.

Appendix A — Engineering corrections implemented in v6.2

- First-token and last-token pooling now resolve positions from `attention_mask`, so left padding cannot silently select padding tokens.
- Preflight now estimates both loaded parameter memory and the all-layer hidden-state activation footprint for the frozen batch.
- Completion provenance now includes dataset head/tail hashes and the native dataset fingerprint when available.
- Runtime anomaly counting records the analysed batch identity so the same batch is not counted repeatedly by periodic reports.
- Runtime state remains a dedicated operational record rather than being replaced wholesale by final extraction metadata.
- External-storage setup is no longer forced during module import; the public experiment entry points initialise it explicitly.
- Model-registry validation now checks names, groups, parameter counts and batch hints, and entries 10–12 have corrected memory-aware frozen batch hints.
- Hyperparameter validation now rejects unsafe experiment IDs and invalid CPU thread counts.

Appendix B — concise interpretation of the present module

In one sentence: Extraction6 v6.2 is a deterministic, provenance-aware, forensic hidden-state acquisition system whose principal output is a layer-by-sample representation tensor for downstream classifier probing.